Who says what

	SPEAKER	COVERS	TIME
1	**Nidhi** Opens the talk	1 Introduction, 2 Problem statement	0:00 to 1:10
2	**Saadhika** Explains the method	3 Methodology, 5 References (one line)	1:10 to 2:00
3	**Diya** Presents the results	4 Results	2:00 to 2:50
4	**Mahiyu** Runs the demo, part one	Demo, steps 1 to 4 (laptop)	2:50 to 3:55
5	**Rajat** Demo part two and the close	Demo, steps 5 to 7, then the close	3:55 to 5:00

Five minutes in total. Everyone speaks for about a minute, and each part ends by naming the next speaker, so the hand-over is part of the script.

Scripts are the team's original speaker notes, split by person with hand-over lines added. The demo is offline, so say so first. The deck itself still lists the five team members alphabetically with equal credit, as the handoff asks.

Opens the talk, *Nidhi*

1 INTRODUCTION, 2 PROBLEM STATEMENT · 0:00 TO 1:10

SLIDE 1, INTRODUCTION

We are Aditi, Diya, Mahiyu, Nidhi and Rajat, and our project is Modula. It is inspired by Google's Project Ara, where you build a phone from swappable modules. You describe how you use your phone, and the system picks the modules and decides where each one goes. The phone on the right is an actual output for a wedding photographer: a large sensor, a large battery and 2 TB of storage for raw files.

SLIDE 2, PROBLEM STATEMENT

A normal phone gives everyone the same compromise. Modular phones promised to fix that, but they move the work to the buyer. Even our small catalogue of 23 modules gives 46,656 combinations, and only 16,420 physically fit on the frame. The bars are to scale. Parts also interact, so a bigger battery adds hours and weight. Google suspended Project Ara in 2016, and reports pointed to cost, battery life and slower links between parts. We can't solve the hardware in a class project, so we looked at a different gap: helping someone choose. Saadhika will now take you through how we built it.

Explains the method, Saadhika

3 METHODOLOGY, 5 REFERENCES (ONE LINE) · 1:10 TO 2:00

SLIDE 3, METHODOLOGY

The system is written in Python, with NumPy for the vector search. The language replies behind our results were recorded earlier, from a first version of the page, and the app plays them back. We split the work in two. Claude handles language: it reads the person and explains the result, though in the offline demo that explanation comes from a template. Python handles everything that has to be correct. The need and every candidate phone sit in the same nine-axis vector space. We filter by space, budget and dependencies, rank what's left, then place the modules using heat, antenna and grip rules. Cosine alone would reward a weak phone with the right shape, so we add attainment, with needs squared so top priorities dominate. We compared three prompts, v1 to v3, as listed on the slide. Diya will show what we found.

SLIDE 5, REFERENCES (ONLY IF ASKED)

Our sources are on the last slide: the studies on Project Ara and Fairphone, the embedding papers behind our vector approach, and two 2026 papers, one on a smartphone recommender that uses embeddings and one on pairing a language model with a constraint reasoner.

Presents the results, *Diya*

4 RESULTS · 2:00 TO 2:50

SLIDE 4, RESULTS

We tuned on 8 personas, then tested once on 16 we had held out. A failed build means no build at all, over budget, or missing a module the person needs. One fixed phone failed 7 of 16. V1 returned clean JSON only 19% of the time and over-specified modules, which made 2 personas impossible to build. V2 scored slightly higher on fit, 0.77 against 0.76, but it caught only 13% of required modules and left 2 people without hardware they needed, which partly explains its higher fit. V3 caught all of them with no failed builds, though it added one module nobody strictly needed. Keyword rules also had no failures, but we wrote that list with these personas in view, so we treat that result with caution. The limits are on the slide. The next step is testing with real users. Mahiyu will now run the demo.

Runs the demo, part one, Mahiyu

DEMO, STEPS 1 TO 4 (LAPTOP) · 2:50 TO 3:55

BEFORE YOU START

Start the app before class: py -m streamlit run app.py.

SAY FIRST

This demo runs offline, so the examples use Claude's recorded replies, the same ones behind the results, while the search and placement run live, and the explanation on screen comes from a template.

STEP 1

Click Wedding photographer and show the phone and chart.

STEP 2

Click Trek guide: the phone rebuilds with a satellite radio and armour, though neither word was typed.

STEP 3

Click Hinglish gamer: "phone garam nahi hona chahiye" means the phone shouldn't heat up, so a cooling module appears.

STEP 4

Open See what the model returned to show Claude's JSON. Then hand over: Rajat will show what happens when you change the priorities and type your own.

Demo part two and the close, *Rajat*

DEMO, STEPS 5 TO 7, THEN THE CLOSE · 3:55 TO 5:00

STEP 5

Open Change the priorities and move the Camera slider to show the phone change.

STEP 6

Type: I shoot videos for YouTube and edit on the phone, need lots of storage, around 60k. Click Build my phone. The status line says keyword matching, no AI, and the build suits the request because those words are on the list.

STEP 7

Type: I fix wind turbines and climb towers in storms. Click Build my phone. Keyword matching finds nothing it knows and says so. That gap is why we use a language model, and why we treat the keyword score with caution.

CLOSE

Claude understands the person, Python makes sure the phone can be built, and together they make a modular product easier to choose. Thank you.